

Lost or Wrong?

A runtime router that tells you *how* a coding agent is failing — and therefore what to do

Ziheng Yu Xuhao Chen

September 13, 2026

Abstract

When a coding agent starts to struggle, the practical decision is: **help it find the code, or make it check its own fix?** Those are opposite actions. Today’s runtime monitors cannot choose between them — they detect *that* an agent is struggling and carry no signal about *how*. On identical rows and folds the published stagnation- and loop-detection families score **0.554–0.607** at predicting failure, and at the question that decides the intervention — is the agent LOST (has not found the code) or WRONG-FIX (found it, and the change does not work) — they score **at or below chance (0.41–0.54)**.

We build the missing piece. **Route** is a causal, reference-free runtime router that reads only the first 10–60% of a run and outputs both whether the run will fail and which failure mode it is in, with the intervention that follows. Trained on one set of runs and tested on **two disjoint sets it never saw** (24 cells: every ordered pair of shard sets $\times$ four prefix fractions) it reaches **0.7145** at failure prediction and **0.7324** at mode discrimination — **+0.156 and +0.136 AUC over the published detector family** on the same rows and folds. Cross-set performance equals within-set performance, so the margin is not fitted to one corpus.

Two results make it trustworthy rather than merely accurate. Its **false-alarm rate is genuinely controlled** ($\alpha = 0.05 \rightarrow 0.046$ achieved, 8 of 8 levels) against an explicit null — replacing an earlier claim of ours that was **circular** and is withdrawn. And as a decision rule it beats every fixed policy: **0.764–0.800** against 0.732 for always-verify, 0.656 for random and 0.518 for always-search, in **12 of 12** cells.

Testing the premise live produced something uncomfortable: on our benchmark suite **100% of runs reached the correct file**, because the verifier prints the failing test’s filename, which contains the module name — localisation was being solved by string-matching, not diagnosis. We report what happens when the location is withheld, including two harness faults that produced numbers looking exactly like findings.

Keywords: coding agents, runtime monitoring, failure modes, intervention, reproducible research.

1 The intervention problem

A coding agent runs for fifty steps and is not solving the task. Someone must decide what to do. The literature offers many ways to notice this — loop detectors, redundancy measures, stagnation triggers, output-shape anomalies — and they are decent at noticing: published detectors sit around AUC 0.6–0.7 [1], and a failure-as-a-process monitor reaches 82% precision at a 2–3% false-alarm rate [2].

But noticing is not deciding. There are two very different reasons an agent can be stuck:

- **LOST** — it has not found the code that needs changing. Long searching, wrong-file edits, or no edit at all. The remedy is to supply a location.
- **WRONG-FIX** — it found the right code and its change does not work. Repeated edits to the same place, tests still failing. The remedy is to force it to re-examine its diagnosis.

These remedies are *opposite*, and applying the wrong one wastes the budget you were trying to save. A monitor that cannot separate them cannot be acted on, however good its AUC.

We measured the two modes on 236,137 mechanically labelled edits across 25,681 runs on 1,213 real software-engineering instances, then replicated on two further disjoint sets of $\sim 26,000$ runs each:

The 67.0 / 69.0 / 66.4% is stable to within 2.6 points across the three disjoint sets, and across model scale (66.8% Llama-70B, 70.7% 8B, 70.2% 405B).

failure mode	definition (mechanical, no judgement)	share
WRONG-FIX	failed, but it <i>did</i> edit a file the gold patch touches	67.0 / 69.0 / 66.4%
LOST	failed and never edited such a file	33.0 / 31.0 / 33.6%

Table 1: Two-thirds of failures happen with the agent already in the right file. Stable to within 2.6 points across three disjoint shard sets, and across model scale (66.8% Llama-70B, 70.7% 8B, 70.2% 405B).

2 A trap we hit first

Before building anything we tried to reproduce the natural measurement of localisation and got what looked like a headline: **failed runs localise better than successful ones** (0.670 vs 0.594 , $p = 3.3 \times 10^{-4}$). It was wrong. That statistic is computed against **the run’s own final patch**: both the numerator’s file set and the denominator come from the run itself, so it measures convergence on what the run produced, not aim at the *correct* file — and a run that fixates on the wrong file and never wavers scores a perfect 1.0.

Recomputed against an independent gold patch for 927 of the 1,213 instances the direction reverses, and does so again on both held-out sets (0.492 vs 0.421 , $p = 3.6 \times 10^{-3}$, $n = 198$; 0.512 vs 0.415 , $p = 2.8 \times 10^{-5}$, $n = 229$ — paired within-instance tests). The control that settles it: restricted to runs whose own patch is a single file that *is* a gold file — where the two target sets must coincide — the metrics agree (0.615 vs 0.638 , mean absolute gap 0.033 over 8,801 runs).

	frozen (0–3)	held-out A (4–7)	held-out B (8–11)
on-target, own patch (<i>the trap</i>)	0.594 / 0.670	0.593 / 0.677	0.614 / 0.686
on-target, gold patch	0.477 / 0.407	0.492 / 0.421	0.512 / 0.415
ever reached a gold file	0.982 / 0.670	0.964 / 0.689	0.976 / 0.664

Table 2: Solved / failed, on three disjoint sets of runs (each $\sim 26,700$ runs, 1,213 shared instances). The trap inverts in all three; the independent target reverses it in all three.

Every quantity in this paper is built on the independent target. We tested three candidate explanations for the inversion (confident wrongness, patch breadth, fixation) and **all three failed**; the mechanism is recorded as open rather than asserted.

3 The system

3.1 What it does

`python demo/route.py -replay <run_id>` reads a run’s first f fraction of steps ($f = 10, 20, 40, 60\%$) and prints, at each fraction, $P(\text{fail})$, $P(\text{wrong-fix} \mid \text{failing})$, and the resulting intervention. On a real held-out run:

```

at 20% of the run:
P(fail) = 0.772   P(wrong-fix | this is a failing run) = 0.682
-> VERIFY (it is in the right place, make it check its fix)
ground truth:  reward=0 -> WRONG-FIX

```

3.2 What it may look at

The design constraint that makes the result meaningful is **causality**. The prefix is steps $0 \dots L - 1$ with $L = \text{round}(f \cdot n_{\text{steps}})$; the run’s own length appears in *no* feature. A self-test overwrites every post-cutoff step and asserts all 112 feature columns are unchanged — it caught a genuine leak during development, now fixed. Labels come from the dataset’s gold patch, never from the agent’s output. Folds are task-disjoint.

The inputs are ordinary observables: edit counts and rates, no-op edit fraction, repeat and entropy of command signatures, the read/edit/run/verify mix, observation-size trend, and the pass/fail/error signals visible so far. Two heads — $P(\text{fail})$ and $P(\text{wrong-fix} \mid \text{fail})$ — and the rule is: intervene only if $P(\text{fail})$ is high, then SEARCH if the mode head says LOST, VERIFY if it says WRONG-FIX.

3.3 One detail that changed a comparison

An earlier version compared the router against “position” and found it barely won. That comparison was **invalid**: at a fixed prefix fraction, position *is* the run’s length — a quantity no online system can know (verified: its AUC matches run length’s to ≤ 0.006). Against a length-matched control the learned model holds at 0.63–0.76 and strictly dominates run length; adding position to the features changes the result by ≤ 0.002 .

4 Results

4.1 The published detectors cannot make this decision

target	Route	position	AgentStop-style	loop / redundancy
will this run fail?	0.7145	0.6718	0.5589	0.45–0.58
LOST or WRONG-FIX?	0.7324	0.5993	0.5960	0.41–0.54

Table 3: Identical rows, identical task-disjoint folds. Averaged over 24 cells (every ordered pair of the three shard sets $\times$ four prefix fractions). On the mode question the published families carry *no information*.

4.2 It transfers to runs it has never seen

trained on	tested on	failure prediction	LOST vs WRONG-FIX
shards 0–3	shards 4–7	0.735	0.749
shards 0–3	shards 8–11	0.749	0.776
shards 4–7	shards 0–3	0.729	0.709
shards 8–11	shards 4–7	0.771	0.770

Table 4: Cross-set AUC equals within-set AUC. **Gain over the field: +0.156 AUC (failure), +0.136 AUC (mode)** — at every fraction, in every direction, on data the model never saw.

4.3 Calibration, done properly

Our own earlier “zero false alarms at every budget” claim was **withdrawn as circular**: the detector and the label were the same statistic, so the numbers reported were literally an oracle’s. This version uses an explicit null (for the mode target, runs that are LOST; for failure, runs that succeed) and a sequential rule valid under arbitrary dependence:

level	achieved false-alarm rate	detection
$\alpha = 0.05$	0.046 (worst split 0.061)	0.160
$\alpha = 0.10$	0.078	0.258
$\alpha = 0.20$	0.123	0.386

8 of 8 levels controlled on the mean. A raw 0.5 threshold flags 98.4% of never-failing runs; the calibrated rule flags **3.39%**. We also report a negative: a formal sequential e-value test is valid and **never fires** at $\alpha \leq 0.05$.

policy (on runs that failed)	value ($\lambda = 0.25$)
Route (router-guided)	0.764–0.800
always VERIFY	0.732
random	0.656
always SEARCH	0.518

Table 5: Beats all three at every fraction and every cost setting — 12 of 12 cells.

4.4 It is worth acting on

5 Testing the premise live: the verifier was giving the answer away

We ran the premise against live agents (DeepSeek V4.1 Flash, temperature 0, real Python packages, their own test suites as verifier, no Docker). Total spend \$0.92.

The first surprise: **every run in both arms reached the correct file** — 100%, which would mean LOST does not exist in this setting. The second surprise was the explanation. pytest prints the failing test’s *filename*, and that filename contains the module name (`test_ioutils.py` → `ioutils.py`), so the defect’s location is readable off the failure message. **51.5% of test observations handed to the agent literally name the gold module.** Localisation in these benchmarks is string-matching, not diagnosis.

We therefore added a **masked** condition: the agent is told *that* tests fail (it still sees “3 failed, 516 passed”) but never *which* or *where*. The grader always sees the true output, so the success criterion is unchanged.

condition	what the agent sees	success	reached the correct file
hinted	the exact file and function	0.550 ($n = 40$)	1.000
unmasked	full pytest output (normal CI)	0.372 ($n = 43$)	1.000
masked	only “N failed, M passed”	<i>see note</i>	<i>see note</i>

Table 6: Valid episodes only. 13 of 96 episodes were dropped because the mutated package did not actually fail its suite at episode start, which makes success meaningless; including them gives 0.596 and 0.449, i.e. the filter *lowers* both arms. The hinted–unmasked difference is **not significant** (Fisher $p = 0.126$).

*Note: the masked condition’s first run produced 0/48 success and 0% reaching the gold file. That was a **harness fault, not a result**: every episode died before its first tool call because the recorded Python interpreter had been deleted from a temp directory between runs. No tokens were spent. It is recorded because the number has exactly the shape of a spectacular finding and had to be checked against the transcripts to be disbelieved. The environment has been rebuilt in-repo and the condition re-run; the corrected numbers are in `results/live/episodes_masked.jsonl`.*

Harness faults we had to fix, because they looked like results. (i) **65.2% of turns were unparseable** in the first pilot — our protocol was ad-hoc text, but the model emits its own XML tool-call format regardless; native function-calling took this to 0–4%. (ii) **The agent was writing its own tests into tests/**, which pytest collects, so an agent-authored passing test could have scored a run as successful *without the defect being fixed*; the verifier is now restored to the packages’ originals before grading, and in 71 of 83 valid episodes the agent had modified test files (1,048 files removed). (iii) the deleted interpreter, above.

6 The demo

```
python demo/route.py -fit # train on shards 0-3, cache the model
python demo/route.py -summary # router vs baselines on held-out shards
python demo/route.py -list # held-out runs with their true modes
python demo/route.py -replay <run_id> # step through one run, turn by turn
```

Offline, deterministic, no API key, and it reproduces the held-out numbers on demand.

7 Limitations

- **The cross-scaffold limit is severe.** Every mechanical measurement rests on a structural footer SWE-agent prints into its observations (`[File: ... (N lines total)]`) in 95.8% of edit steps. That footer is **absent from every other scaffold tested** — OpenHands, the PI agent, mini-swe-agent-plus and a multi-framework corpus all show 0%. Cross-framework generality is **not claimed**.
- **The dead-end rate is scaffold-specific, not a property of coding agents:** 3.7% to 23.6% across four other scaffolds — a factor of six. 19.1% is SWE-agent’s number.
- **The 3.7% figure rests on patch recovery from the transcript** for a scaffold with no printed patch; reported as a discrepancy to investigate.
- **The mode labels are coarse.** “Ever touched a gold file” is a proxy, not a judgement of fix quality; gold matching is basename-based and 4.10% of edit steps carry no filename, which biases LOST upward.
- **The live experiment is small** ($n = 40\text{--}49$ per arm). The hinted-vs-unmasked difference is **not significant** (Fisher $p = 0.126$) and is reported as such.
- **The inversion’s mechanism is unexplained:** three candidates tested, all failed.
- **Temperature-0 inference is not deterministic** ($\sim 9\%$ of per-instance outcomes flip across repeats), so within-instance comparisons are reported with runs-per-instance (mean 21.9).
- **The corpus is 80,035 runs across three disjoint tables**, not one merged table. The single-table rebuild was attempted, failed, and the failure is recorded.

8 Reproducibility

Six automated gates run on every change: invariant tests 11/11; cross-artifact consistency passes; 28/28 headline claims trace to their artifacts; 0 mismatches against the one-page summary; every cited artifact exists; and 17/17 paper numbers trace to the artifact each one names. That last gate was written during this work and **found four real errors in our own manuscript** on its first run — values quoted as pooled that were within-instance, and figures mixed across two artifacts’ row sets. All are fixed. Complete data package: `research/EXPORT/`.

9 Use of AI tools

A language model (`deepseek-v4.1-flash`) was used as a research assistant throughout — writing analysis code, running statistical tests, retrieving and verifying literature, and drafting text. Every load-bearing citation was then verified by hand against the primary source, and two reconnaissance claims were rejected as unverifiable. The model also served as the *subject* of the live experiment. It was not used to generate data; all measurements come from public corpora and real test suites executed locally. **Eight of its own claims were falsified and retracted**, documented pass by pass in the project’s assistance log. Full disclosure, including the required chat records, is in the separate acknowledgement and disclosure document.

References

- [1] Pham et al. *Terminating Local AI Agents Early to Save Energy in Consumer Devices*. arXiv:2605.15206.
- [2] *Failure as a Process*. arXiv:2607.09510.
- [3] *RedundancyBench*. arXiv:2605.29893.
- [4] Mehta et al. *Coherence Collapse: Diagnosing Why Code Agents Fail After Reaching the Right Code*. arXiv:2603.24631.
- [5] Mehta. *Confident and Wrong: Silent Semantic Failures in Coding Agents*. arXiv:2603.25764.

- [6] *Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories.* arXiv:2511.00197.
- [7] *FailForge.* arXiv:2608.08570.
- [8] *TRIM: Reducing AI-Generated CodeSlop via Agent Trajectory Minimization.* arXiv:2607.18161.
- [9] Zhu et al. *Which Agent Causes Task Failures and When?* arXiv:2505.00212.
- [10] *Temperature-0 nondeterminism on SWE-bench Verified.* arXiv:2607.09691.